CollegeQuery - Multi-College AI Information and Query Platform

An AI-powered platform where students can search colleges, view verified profiles, and ask natural-language questions about each college. Answers are grounded in verified college data and come with source citations.

Author: Piyush Jha

Project Type: Internship Project

Domain: AI / Retrieval-Augmented Generation (RAG) / Full-Stack Web Development

Status: Phase 0 and Phase 1 completed - Phase 2 in progress

Table of Contents

1. Project Overview
2. Objectives
3. Scope
4. Key Features
5. Technology Stack
6. System Architecture
7. Multi-College Data Isolation
8. Database Design
9. RAG Pipeline
10. API Design
11. Repository Structure
12. Implementation Roadmap
13. Testing Strategy
14. Deployment Plan
15. Getting Started
16. Risks and Mitigations
17. Future Enhancements
18. Author

---

1. Project Overview

Students often struggle to find accurate, up-to-date information about colleges. Details about admissions, fees, hostels, placements, and facilities are scattered across brochures, PDFs, and websites.

CollegeQuery solves this by providing one platform where:

- Every college has its own verified profile and document set.
- Students ask questions in plain English, for example *"What is the hostel fee at College A?"*
- The system retrieves the relevant passages from that college's documents only, generates an answer with an LLM, and cites the source documents.

CollegeQuery is **not a single-college chatbot**. It is designed from day one as a **multi-college platform**, where all data is tied to a unique `college_id`.

---

2. Objectives

- Build a working multi-college question-answering platform using RAG.
- Ensure every answer is grounded in verified data and shows its sources.
- Prevent any mixing of data between colleges.
- Deliver a clean MVP first, then expand in controlled phases.
- Follow professional engineering practices: migrations, tests, documentation, and clean architecture.

---

3. Scope

3.1 MVP (In Scope)

Area	Included
Colleges	3 colleges initially
Search	College listing, search, and profile pages
Chat	Ask questions per college with cited answers
Ingestion	Admin PDF upload, text extraction, chunking, embeddings
Admin	Admin login (JWT) and college/document management
Topics	Admissions, courses, departments, fees, hostel, campus, placements, facilities, academics, notices

3.2 Out of Scope for MVP

- Automatic website crawling
- College comparison
- Student accounts and chat history
- Analytics dashboard
- Microservices, Kubernetes, Kafka, Redis, Celery

---

4. Key Features

- **College directory** - search and browse colleges with detailed profiles.
- **College-specific Q&A** - ask a question inside one college's context.
- **Source citations** - every answer lists the document and page it came from.
- **Safe fallback** - if the answer is not in the data, the bot says so instead of guessing.
- **Admin panel** - add colleges, upload PDFs, and track ingestion status.
- **Secure admin access** - JWT-based authentication for admins only.

---

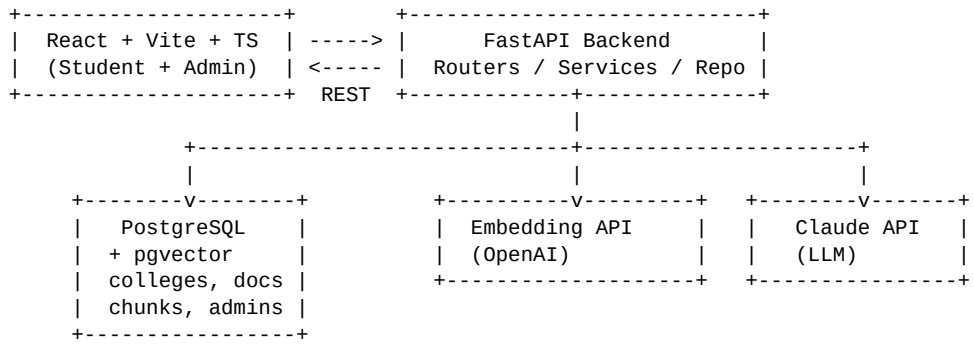
5. Technology Stack

Layer	Technology	Why this choice
Frontend	React + Vite + TypeScript	Fast dev server, type safety, large ecosystem
Styling	Tailwind CSS	Rapid, consistent, responsive UI
Backend	FastAPI (Python)	Async, auto-generated API docs, strong AI/ML ecosystem
Database	PostgreSQL	Reliable relational store for structured college data
Vector Search	pgvector	Vectors live in the same DB, so no extra service to run
ORM	SQLAlchemy	Mature, flexible, works well with FastAPI

Layer	Technology	Why this choice
Migrations	Alembic	Versioned, repeatable schema changes
PDF Extraction	PyMuPDF	Fast and accurate text extraction with page numbers
Embeddings	OpenAI Embeddings	High-quality semantic vectors, simple API
LLM	Claude API	Strong grounded answering with clear instruction following
Auth	JWT (admins only)	Simple and stateless for the MVP
Containers	Docker Compose	One-command local setup for Postgres + pgvector

---

6. System Architecture



Backend layering: `Router -> Service -> Repository -> Database`

- **Routers** handle HTTP and validation (Pydantic schemas).
- **Services** hold business logic and the RAG pipeline.
- **Repositories** handle all database access (SQLAlchemy).

---

7. Multi-College Data Isolation

This is the most important design rule of the project.

- Every row in `documents` and `chunks` carries a `college_id`.
- Every retrieval query includes `WHERE college_id = :college_id`.
- The chat endpoint **requires** a `college_id`; requests without it are rejected.
- Prompts sent to the LLM contain context from **one college only**.
- Tests verify that a question about College A never returns College B data.

---

8. Database Design

Table	Purpose	Key columns
`colleges`	College profile	id, name, slug, city, state, website, description, created_at
`departments`	Departments per college	id, college_id, name, description
`documents`	Uploaded source files	id, college_id, title, source_type, file_path, status, created_at
`chunks`	Text chunks with vectors	id, college_id, document_id, content, page_number, embedding (vector 1536)
`admins`	Admin accounts	id, email, password_hash, created_at

Notes

- Indexes on `college_id` for all tenant-scoped tables.
- A vector index (HNSW or IVFFlat) on `chunks.embedding` for fast similarity search.
- All schema changes are tracked using Alembic migrations.

---

9. RAG Pipeline

9.1 Ingestion (Admin uploads a document)

1. Admin uploads a PDF for a specific college.
2. PyMuPDF extracts text page by page.
3. Text is cleaned and split into overlapping chunks (about 800 tokens, 100 overlap).
4. Each chunk is embedded using the embedding model.
5. Chunks are stored with `college_id`, `document_id`, and `page_number`.
6. Document status moves from `processing` to `ready` (or `failed`).

9.2 Query (Student asks a question)

1. Student selects a college and asks a question.
2. The question is embedded.
3. pgvector returns the top-k similar chunks **filtered by college_id**.
4. A prompt is built from the question and the retrieved chunks.
5. Claude generates an answer using only the provided context.
6. The API returns the answer plus citations (document title and page).
7. If no relevant context is found, a safe fallback message is returned.

---

10. API Design

Base path: `/api/v1`

Method	Endpoint	Access	Description
GET	`/health`	Public	Service health check
POST	`/auth/login`	Public	Admin login, returns JWT
GET	`/colleges`	Public	List and search colleges
GET	`/colleges/{slug}`	Public	College profile
POST	`/colleges`	Admin	Create college
PUT	`/colleges/{id}`	Admin	Update college
DELETE	`/colleges/{id}`	Admin	Delete college
POST	`/colleges/{id}/documents`	Admin	Upload and ingest PDF
GET	`/colleges/{id}/documents`	Admin	List documents and status
DELETE	`/documents/{id}`	Admin	Remove document and its chunks
POST	`/chat`	Public	Ask a question for one college

Chat request / response

```
{ "college_id": 1, "question": "What is the hostel fee?" }

{
  "answer": "The annual hostel fee is ...",
  "citations": [{ "document": "Fee Structure 2025", "page": 4 }]
```

```
}
```

11. Repository Structure

```
collegequery/
|-- backend/
|   |-- app/
|   |   |-- api/            # Routers
|   |   |-- core/           # Config, security, logging
|   |   |-- models/         # SQLAlchemy models
|   |   |-- schemas/        # Pydantic schemas
|   |   |-- repositories/   # DB access layer
|   |   |-- services/       # Business logic + RAG
|   |   `-- main.py
|   |-- alembic/            # Migrations
|   |-- scripts/            # Seed data
|   |-- tests/
|   `-- requirements.txt
|-- frontend/
|   |-- src/
|   |   |-- components/
|   |   |-- pages/
|   |   |-- services/       # API client
|   |   `-- main.tsx
|   `-- package.json
|-- docs/                   # Architecture and design notes
|-- docker-compose.yml
|-- .env.example
`-- README.md
```

12. Implementation Roadmap

Phase	Title	Key deliverables	Status
0	Project setup	Repo scaffold, FastAPI health endpoint, React shell, Docker Compose (Postgres + pgvector), env examples	Done
1	Database foundation	SQLAlchemy models, Alembic initial migration, seed script with 3 colleges	Done
2	Colleges API	CRUD endpoints, repository and service layers, validation, tests	In progress
3	Admin auth	Admin model, password hashing, JWT login, protected routes	Planned
4	Document ingestion	PDF upload, PyMuPDF extraction, chunking, embeddings, storage	Planned
5	Retrieval and chat	College-filtered vector search, prompt building, Claude answers, citations	Planned
6	Student frontend	College list/search, profile page, chat interface with citations	Planned
7	Admin frontend	Login, college management, document upload and status	Planned
8	Testing and hardening	Isolation tests, error handling, rate limiting, logging	Planned
9	Deployment	Dockerized deployment, environment config, live demo	Planned

13. Testing Strategy

- **Unit tests** - services, chunking logic, prompt builder.
- **Integration tests** - API endpoints against a real Postgres + pgvector instance.
- **Isolation tests** - confirm no cross-college retrieval.

- **Answer quality checks** - a small set of reference questions per college to verify grounded answers and correct citations.
- **Frontend checks** - build and lint on every change.

Current status: backend tests (14/14) pass for Phase 0 and Phase 1.

---

14. Deployment Plan

- Containerize backend and frontend with Docker.
- Managed PostgreSQL with the pgvector extension.
- Environment variables for API keys and secrets (never committed).
- HTTPS and CORS configured for the production domain.
- Basic logging and health checks for monitoring.

---

15. Getting Started

Prerequisites: Python 3.11+, Node.js 18+, Docker, Git

```
# 1. Clone the repository
git clone https://github.com/<your-username>/collegequery.git
cd collegequery

# 2. Start PostgreSQL with pgvector
docker compose up -d

# 3. Backend setup
cd backend
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env            # add your API keys
alembic upgrade head
python -m scripts.seed          # load sample colleges
uvicorn app.main:app --reload

# 4. Frontend setup (new terminal)
cd frontend
npm install
cp .env.example .env
npm run dev
```

- Backend API docs: `http://localhost:8000/docs`
- Frontend: `http://localhost:5173`

---

16. Risks and Mitigations

Risk	Mitigation
LLM gives incorrect or invented answers	Strict context-only prompt, citations, and safe fallback message
Data leaking between colleges	Mandatory `college_id` filter and dedicated isolation tests
Poor PDF text quality	Cleaning step, review of extracted text, manual re-upload option
API cost growth	Small chunk counts (top-k), caching later, usage monitoring
Scope creep	Fixed MVP boundaries and phase-by-phase delivery

---

17. Future Enhancements

- Automatic website crawling and scheduled re-ingestion
- Side-by-side college comparison
- Student accounts and saved chat history
- Analytics dashboard for popular questions
- Multi-language support
- Re-ranking and hybrid (keyword + vector) search

---

18. Author

Piyush Jha

B.Tech, Information Technology

Institute of Aeronautical Engineering (IARE), Hyderabad

If you find this project useful, consider giving it a star on GitHub.